Substrate as Primary Source of State for the LLM Mediator: Property A as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 2 May 2026

— — —

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five mediator properties named in the source paper's AI-as-substrate-mediator commitment — **Property A: the LLM reads from substrate content as its primary source of state** — as a standalone architectural commitment with independent operational content, separable from the four other properties (B, C, D, E) with which it composes into the integrated mediator role.

Abstract

The CKS pattern's AI-as-substrate-mediator commitment names five properties that jointly define the LLM's role within cells (§4.2). A parent-frame note (A2.18) establishes that the five are individually severable architectural commitments. This note formalizes the first — Property A: the LLM reads from substrate content as its primary source of state — as having independent operational content, separable from how Properties B, C, D, and E are realized. The motivation is concrete: implementations of CKS-shaped systems consistently drift toward primary input sources other than the substrate (agent-framework context windows, retrieval indexes, chat turn history, operator prompts), producing systems where the substrate exists but is not authoritative. The note states Property A's four operational components, distinguishes it from four adjacent architectures, identifies seven failure modes that violate it specifically, and provides an operational test for whether a system's LLM input behavior is CKS-coherent on this axis.

1. Why Property A needs to be formalized as standalone

The CKS pattern's AI-as-substrate-mediator commitment names five properties that jointly define the LLM's role within cells (§4.2; parent foundational note A1.04). The parent-frame note A2.18 establishes that the five — Property A (substrate reads as primary input), Property B (writes under orchestration rules), Property C (no substrate-relevant state outside the substrate), Property D (no authority over substrate content), Property E (recorded with attribution) — are not a single composite. Each is an independent architectural commitment, and a system can satisfy some while failing others.

Implementations of CKS-shaped systems consistently drift toward four primary-input substitutes: agent-framework context windows that hold state across multi-step reasoning; chat-application turn history;

retrieval indexes ranked alongside substrate as one corpus among several; and operator prompts that embed coordination state directly. Each pattern produces the same architectural failure: the LLM's primary input is not the substrate, so the LLM's coordination conclusions are not grounded in what the substrate carries. The substrate exists and may be written to under appropriate authority, but it is not the authoritative source the LLM consults; substrate writes flow from reasoning that bypassed substrate reads.

Two further motivations support the standalone treatment. First, patentable derivations of CKS that focus on LLM input architectures — context-window design, retrieval configuration, prompt-substrate integration — are more defensibly contested when Property A is publicly formalized as standalone, because any candidate "input innovation" can be evaluated against the explicit architectural commitment. Second, the source-of-truth commitment (A1.08; §11.3) depends operationally on Property A: substrate-as-source-of-truth is satisfied at the storage layer but undermined at the read layer when the LLM's primary input is something other than substrate.

2. The Property A commitment, defined precisely

In the CKS pattern, an LLM operating within a cell satisfies **Property A** if and only if all four of the following hold during the cell's execution.

(a) Substrate as authoritative input for coordination questions. When the LLM's execution must determine what is the case for a coordination question — what was decided, by whom, under what authority, with what rationale, what conflicts remain — the LLM consults substrate content for the answer. Substrate is authoritative for the §11.3 categories; non-substrate inputs are not.

(b) Direct read in substrate-preserving form. The LLM reads substrate content through the substrate→cell read crossings the architecture commits to (treated at the boundary layer in A2.10), in a form that preserves the substrate's authoritative meaning. Transformations that abstract away from substrate state — collapsed contradictions, removed provenance, smoothed ambiguity — do not satisfy what "primary input" means.

(c) Authoritative status against supplementary inputs. When the LLM has access to multiple input sources — substrate plus retrieval, substrate plus tool outputs, substrate plus operator prompts — substrate content is authoritative for what the substrate carries. Supplementary inputs are informative but cannot override substrate authority on coordination questions. The architecture's posture is asymmetric on purpose: substrate authoritates; supplements inform.

(d) Reasoning grounded in substrate-read content. When the LLM produces outputs that subsequently affect substrate state under Properties B and E, the reasoning behind those outputs traces to substrate content the LLM read. Reasoning grounded in non-substrate sources produces outputs whose antecedents — when traced through Property E's attribution — point outside substrate, which fails path-retraceability (A1.07) for the substrate-affecting work.

A system that satisfies fewer than four of (a)–(d) produces LLM behavior in which the substrate is not the primary authoritative input for coordination questions, regardless of what other architectural commitments the system carries.

3. What Property A does NOT require

Stating what the property does not commit to keeps the standalone framing from drifting beyond what the source paper supports.

Not exclusivity of substrate as input. The LLM may read orchestration-rule content (substrate-resident, per A2.04), prompt scaffolding, tool outputs, retrieval over external source documents, and other supplementary inputs. What the property requires is that substrate is *primary* and *authoritative* for coordination questions. A1.16's Pattern A admits adjacent components as supplementary inputs; Property A is satisfied provided substrate retains authoritative status for what the substrate carries.

Not full-substrate reads every execution. The LLM reads within the cell's bounded scope — the slice of substrate the orchestration rule specifies as relevant for the cell's task (A2.09). Property A is about substrate's authority within scope; it is not a commitment to read everything.

Not synchronous reads. The substrate read may be asynchronous, batched at execution start, or staged across multi-step reasoning. What matters is that substrate is the source the LLM eventually reads from for coordination questions.

Not LLM-direct access to substrate storage. Cell-layer code may fetch substrate content and supply it to the LLM as input; the read need not be performed by the LLM itself. What matters is that the LLM's input *is* substrate content, in a form that preserves substrate's authoritative meaning.

Not a specific format. Substrate may be serialized as structured data, formatted as natural language, or rendered in any other form, provided substrate's meaning is preserved.

4. What Property A is NOT

Property A is most often confused with the four adjacent input architectures listed below. Each is a real and reasonable design choice in some other context; naming what Property A is *not* is what prevents the misreading.

Not retrieval-augmented generation (RAG). RAG retrieves supplementary content from indexes built over source documents, knowledge bases, or other corpora, supplying the retrieved content to the LLM as grounding; retrieval ranking determines which content "wins" as input. Property A is different in two respects. First, substrate is the LLM's primary *authoritative* input for coordination questions, not a corpus to be retrieved over and ranked against other corpora. Second, substrate's authoritative status is architectural, not the result of relevance scoring. A CKS system can use RAG as an adjacent component for source-document grounding under A1.16's Pattern A — retrieving evidence, prior writings, or other supplementary content — without violating Property A, provided substrate remains authoritative for what the substrate carries (the §11.3 categories). The violation occurs when substrate is treated as one corpus among several in a retrieval pipeline; relevance scoring then determines what the LLM treats as authoritative, and substrate's source-of-truth status becomes ceremonial.

Not agent-framework primary context. Agent frameworks typically position the agent's context window as the primary state holder for multi-step reasoning, with the agent's conclusions grounded in whatever the context carries. Memory across invocations, planning state, tool-output history, and retrieved content all accumulate in context, and the agent reads from context as its primary authoritative input. Property A is different: substrate, not context window, is the primary authoritative source for coordination questions. The context window may carry intermediate reasoning and supplementary inputs the LLM uses within a cell

execution; what it must not carry, treated as authoritative, is substrate-relevant state that should have been read from substrate. A CKS deployment using an agent framework as the cell's execution mechanism must constrain the agent to read from substrate as primary input; using the framework's native context-as-primary pattern violates Property A even when substrate is referenced secondarily.

Not chat-application turn history. Chat applications typically position prior conversational turns as the primary context for subsequent reasoning. Property A is different: substrate, not turn history, is the primary authoritative source for coordination questions. Prior turns may inform the LLM's understanding of what the human asked, but they do not authoritate coordination questions. A CKS system implemented over a chat interface must commit substrate writes between turns under Properties B and E and have subsequent turns read from substrate for coordination conclusions, not simply continue from the prior turn's reasoning.

Not prompt-engineered primary inputs. Prompt-engineering practices often embed authoritative content directly in the prompt — system prompts that carry coordination rules, persona definitions, or operator-asserted facts. Property A is different: orchestration rules and substrate content are substrate-resident (A2.04), not prompt-resident. Prompts may include scaffolding that helps the LLM operate over substrate content; the content the LLM treats as authoritative is substrate content, not prompt content.

The four distinctions share a common shape: a non-substrate source occupies the role Property A reserves for the substrate.

5. Why Property A is load-bearing for downstream commitments

Several other CKS commitments depend on Property A operationally.

Source-of-truth (A1.08; §11.3). Substrate is authoritative for the §11.3 categories only because LLMs operating in cells read substrate as primary input. If LLMs drew their primary input from agent memory, retrieval rankings, or operator prompts, substrate's authoritative status would be a property of storage but not of the system's actual behavior. Property A is what carries source-of-truth from storage into operational practice.

Cell-level conflict resolution (A2.14). Conflicts preserved in substrate are resolved at cell execution time under orchestration rules. The mechanism requires that the LLM encounter the contradiction in substrate content. If the LLM read from a derived view that smoothed contradictions, or from agent memory that resolved them silently across prior interactions, the cell-level resolution mechanism would have nothing to operate on.

Path retraceability (A1.07). Decisions made by LLMs are retraceable through provenance only if the antecedent reference points to substrate content the LLM actually read. If the LLM's reasoning was grounded in non-substrate sources, the retraceable path has antecedents the architecture cannot inspect.

KO/OIDA inheritance (A1.09). The inheritance from external structured memory is operational only when LLMs treat the inherited substrate as authoritative input. If consulted secondarily, the inheritance becomes nominal rather than architectural.

6. Failure modes that violate Property A

The four adjacent architectures named in §4 distinguish Property A at the design-pattern level. The failure modes named here distinguish it at the implementation level — specific operational behaviors a system can exhibit while otherwise carrying CKS shape.

(a) LLM context window as primary state. The LLM holds coordination state in its context window across turns or multi-step reasoning, and consults substrate only when context lacks needed information. This is the most common drift in implementations migrated from chat or agent-framework patterns, because the underlying inference mechanism naturally privileges what is already in context.

(b) Agent memory as primary state. The LLM operates within an agent framework that maintains agent memory across executions; the agent reads from agent memory as primary input and from substrate as supplementary. This is the failure mode the source paper at §6.2 frames as *context rot* — the agent memory degrades through capacity overflow, compaction loss, and goal drift while continuing to serve as primary input, and substrate-derived corrections become increasingly difficult to integrate against the accumulated agent state.

(c) Retrieval results as primary state. The LLM operates over retrieval results from external indexes as primary input, with substrate as one index among many. Coordination questions are answered based on retrieval ranking rather than substrate authority; what the LLM treats as the answer is whatever the retrieval pipeline ranked highest for the query.

(d) Prompt content as primary state. The LLM operates over operator-supplied prompts that contain coordination state or rules embedded in the prompt itself, and substrate is consulted only when the prompt does not specify. This is common in prompt-engineering-heavy deployments that have not migrated coordination state to substrate.

(e) Cached substrate as primary state. The cell maintains a cache of recent substrate reads and consults the cache as primary input, with substrate consulted only when the cache lacks needed content. This breaks Property A even though the cache was originally populated from substrate, because the cache may drift from substrate state without the architecture knowing — "primary" in Property A means the LLM reads from what is currently authoritative, not from what was authoritative at the time of caching.

(f) Summary or transformed representations as primary state. Substrate content is transformed before reaching the LLM — summarized, embedded, normalized into a derived schema — and the LLM reads the transformation as primary input. The transformation may abstract away from substrate state in ways that lose authoritative content (collapse contradictions, drop provenance, paraphrase rationale into something the substrate did not say). This is the architectural concern with derived views (A1.16's Pattern B) treated as primary instead of as derived: derived views are admissible as supplementary, but Property A reserves primary authoritative status for substrate content as the substrate actually carries it.

(g) Operator-asserted facts as primary state. The cell operator asserts facts in the prompt or supplementary inputs that override substrate content. This violates Property A because operator assertions are not substrate content — they may not have been committed, may not carry provenance, and may not reflect what substrate currently carries. The remedy is for the assertion to be committed to substrate first, under appropriate authority, where it becomes substrate content the LLM reads from per Property A.

A system can exhibit one of these failure modes while satisfying the other four mediator properties (B, C, D, E). Such a system fails Property A specifically. Naming the failure precisely is what allows downstream

remediation, because the remedy is targeted at the input layer rather than at the system as a whole.

7. Operational test

A system satisfies Property A if and only if all of the following are true at all times during LLM execution within cells.

1. For coordination questions the LLM's execution addresses (the §11.3 categories: what was decided, by whom, under what authority, with what rationale, what conflicts remain), the substrate is the authoritative input source the LLM consults.

2. The LLM reads substrate content directly, in a form that preserves substrate's authoritative meaning, through substrate→cell read crossings (treated at the boundary layer in A2.10).

3. When the LLM has access to multiple input sources, substrate's authoritative status holds against supplementary inputs for coordination questions; other inputs are informative but not authoritative.

4. The LLM's reasoning that produces substrate-affecting outputs is grounded in substrate content the LLM read — the antecedent reference for the output's provenance points to substrate content, not to non-substrate sources.

A system that fails any of (1)–(4) does not satisfy Property A in the architectural sense, even when its other mediator properties (B, C, D, E) are preserved. Such a system may instantiate some other architectural posture for its LLM input behavior, and may be useful for some other purpose, but is not CKS-coherent on the input axis.

8. Why naming Property A as standalone matters

Implementations of CKS-shaped systems consistently drift toward primary input sources other than the substrate, because the alternative patterns are well-documented, well-tooled, and architecturally familiar; substrate-as-primary is unusual and requires explicit architectural commitment. Implementations that drift produce systems where substrate exists but is not authoritative: substrate is consulted occasionally, but coordination conclusions are formed elsewhere. The downstream consequences are predictable — substrate writes that contradict substrate content, retraceability gaps, and source-of-truth failures.

Naming Property A as a standalone architectural commitment gives downstream implementers a precise specification of what Property A requires architecturally, independent of how Properties B, C, D, and E are realized. The standalone treatment does not contradict the joint commitment of the AI-as-substrate-mediator role; it elaborates one severable property within it. Subsequent notes (A2.20–A2.23) elaborate Properties B, C, D, and E on the same standalone basis.

Subsequent work that implements, extends, composes, or argues against Property A should use the term in the sense formalized here. Subsequent work that grants the LLM a primary input source other than substrate is using a different architectural posture, and the difference should be named.

— — —

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Substrate as Primary Source of State for the LLM Mediator: Property A as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern.* 2 May 2026. ORCID: 0009-0004-8065-3235.